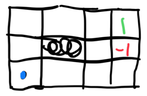
Markov Decision Process



Collect reward
Stochastic transitions

- $Pr(s'|s,a)$: Probability we transition to state s' if we choose action a in state s
- MDP: $Pr(s'|s,a)$, MP: $Pr(s'|s)$

$R(s)$: Reward function
$\pi(s)$: Policy (Example)

$U([s_0, s_1, \dots, s_T])$:= Utility of a sequence of states of like time

$$= \sum_{t=0}^{T} R(s_t)$$

or

$$= \sum \gamma^t R(s_t) \quad \gamma \in (0,1]$$

(Decisions that help u now vs planning for later)

Value Iteration

This leads to

$$U^{\pi}(s) = \mathbb{E}_{Pr([s_0, s_1, \dots]|s_0=s,\pi)}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]$$

r.v. τ ; $G(\tau)$:= return, thing u are averaging

Given all future trajectories given a s_0 and π, what is the average reward?

$\pi_s^* = \arg\max_{\pi} U^{\pi}(s) = \pi^*_{s'}$ for any s'

Max gives you largest value

Bellman Equation: $U^*(s) = \max_a \left[\sum_{s'} P(s'|s,a)\left(R(s,a,s') + \gamma U^*(s')\right)\right]$

Recursive def (how it is solved in practice)

$\arg\max$ gives you input that produced largest value

$\pi^*(s) = \arg\max_a \left[\sum_{s'} P(s'|s,a)\left(R(s,a,s') + \gamma U^*(s')\right)\right]$ (need $U^*(s)$ first)

From U^* u can get π^*, but not vice versa

If policy Stochastic: no one action, but probability of choosing an action

$$U^{\pi_t}(s) = \sum_a \pi_t(a|s) \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma U^{\pi_t}(s')\right]$$

Initial Guesses for π_{t_0} for $\forall s_0 \in S$ ($\pi_{t_0}(s_0) = a_0$ deterministic)

$$U^{\pi_t}(s) = \sum_{s'} P(s'|s,\pi_t(s))\left(R(s,\pi_t(s),s') + \gamma U^{\pi_t}(s')\right)$$

$$\pi_{t+1}(s) = \arg\max_a \sum_{s'} P(s'|s,a)\left(R(s,a,s') + \gamma U^{\pi_t}(s')\right)$$

Value Iteration

U can say $s' = s_{t+1}$ & $s = s_t$ here

Problems

- Computes utility for every state
- Needs exact transition model
- Needs fully observe State
- Needs to know exact reward for each state

Tabular Q-Learning

- $target(s') = R(s,a,s') + \gamma \max_{a'} Q_k(s',a')$
- $Q_{k+1}(s,a) = (1-\alpha)Q_k(s,a) + \alpha[target(s')]$
- How to sample actions?
- $\pi(s^*) = \arg\max_a Q(s^*,a)$
- How to Sample actions?
  - ε-greedy: pick best action given current $Q_k(s,a)$ by ε and random action by $(1-\varepsilon)$
- Counters bad initialization

Partially-observable MDPs

$(S, A, T, R, \Omega, O, \gamma)$ — a_t = action at time t

Hidden:
- States S
- actions A
- T: Transition fct $T(s'|s,a) = P(s_{t+1}=s' | s_t=s, a_t=a)$
- R: Reward fct: $R(s,a,s')$

Observable:
- Observations Ω: Sensor readings and tokens the agent can actually perceive
- Observation fct O: $O(o|s',a) = P(o_{t+1}=o | s_{t+1}=s', a_t=a)$
- Discount factor γ

agent doesn't know exact state s, so maintains belief state $b(s)$

$$\sum_{s \in S} b(s) = 1$$

Belief updates: $b'(s') = \eta \cdot O(o|s',a) \sum_{s \in S} T(s'|s,a)\, b(s)$ (Bayes rule)

normalization: $\eta = \frac{1}{P(o|b,a)}$

Now, we do $\pi(b(s)) \rightarrow$ action a instead of $\pi(s) \rightarrow a$

1. you have b_t
2. Then you have $a_t = \pi_t(b_t)$
3. o_{t+1}
4. $b_{t+1}(s') = \eta O(o_{t+1}|s',a_t) \sum_s P(s'|s,a_t) b_t(s)$
5. $U^{\pi_t}(b_t) = R(b_t, \pi_t(b_t)) + \gamma \sum_{o_{t+1}} P(o_{t+1}|b_t, \pi_t(b_t)) U^{\pi_t}(b_{t+1})$
6. $\pi_{t+1}(b) = \arg\max_a \left[R(b_t,a) + \gamma \sum_{o_{t+1}} P(o_{t+1}|b_t,a) U^{\pi_t}(b_{t+1})\right]$

Policy Gradient Algorithm & PPO

$$J = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \int \pi_\theta(\tau) R(\tau)\, d\tau \qquad \mathbb{E}_{x \sim P(x)}[X] = \int x P(x)$$

Objective

$$\nabla_\theta J = \int R(\tau)\left[\pi_\theta \nabla_\theta \log \pi_\theta(\tau)\right] d\tau = \mathbb{E}_{\tau \sim \pi_\theta}(A(\tau) \nabla_\theta \log \pi_\theta)$$

Reinforce Alg:
1. Reinforce one full trajectory (τ) using π_θ
2. Calc $R(\tau)$ from black box (τ = trajectory/complete response)
3. Calc score: $\nabla_\theta \log \pi_\theta(\tau)$
4. $\theta \leftarrow \theta + \alpha \cdot [R(\tau) \nabla_\theta \log \pi_\theta(\tau)]$

Prob: you ignore good & bad phrases within thing, so need baseline

Value fct: Now Say: $A(s_t, a_t) = R_t - V(s_t)$

Now $\nabla_\theta J = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, A(s_t,a_t)\right]$

Instability tho, i.e massive gradient steps, so we need "Safety harness"

PPO

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \qquad L^{clip}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)]$$

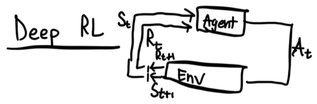
Maximize this

$$\theta \leftarrow \theta + \alpha \nabla_\theta L^{clip}(\theta)$$

1. $A_t > 0$ $\quad \min(r_t(\theta)A_t, (1+\varepsilon)A_t)$ (ceiling) if $r_t > 1+\varepsilon$ objective goes flat (floor)
2. $A_t < 0$; $r_t < 1-\varepsilon$ " "

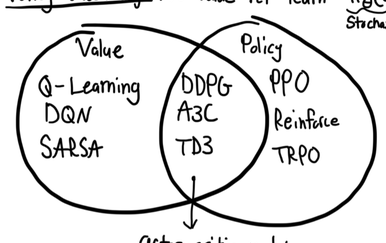
Actor-Critic Model

Deep RL



Value-based Alg: Learn value fct $Q(s,a)$, Implicit π; $\arg\max Q$ (Challenges in continuous action space)

Policy-based alg: No value fct learn $\pi_\theta(a|s)$ (Stochastic) or $\mu_\theta(a|s)$ (deterministic) (High variance)



Acter-critic model (best of both worlds)

For recursive stuff they guess, use A assuming B is right, then fix B, etc.

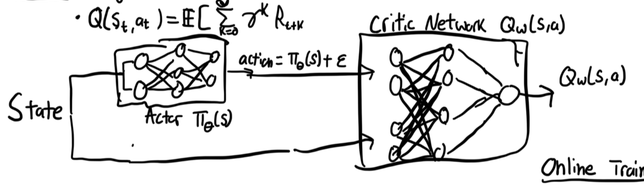
TD: A update, B update → A update; C → B → A

Monte-Carlo: Wait til ending, then go backwards once (only if end episode exists/finite)

Actor: Decides which action to take (Policy-based method)

Critic: How good was action & how to adjust (Value-based)

- $Q(s_t, a_t) = \mathbb{E}[\sum_{k=0} \gamma^k R_{t+k}$



Online Training: One experience at a time

Offline Training: In batches

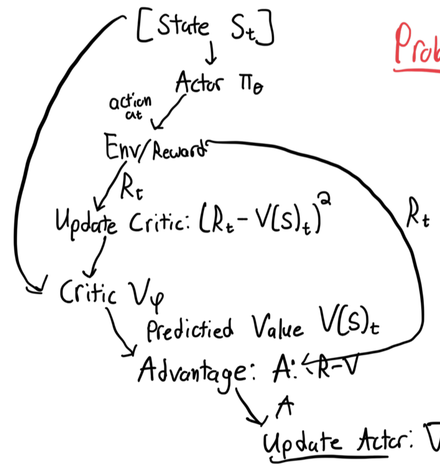
Obtain data experience $[s_t, a_t, r_t, s_{t+1}]$

Training

④: Compute $Q_w(s,a)$

⑤ $\Delta\theta = \underbrace{\alpha}_{\text{learning rate}} \nabla_\theta \log \pi_\theta(s_t,a_t) \underbrace{Q_w(s_t,a_t)}_{\text{Value fct}}$ (Gradient) Acter updates policy params θ using Q-val → Policy grad

⑥ $\Delta w = \beta[\underbrace{R(s_t,a_t) + \gamma Q_w(s_{t+1}, a_{t+1}) - Q_w(s_t,a_t)}_{\delta \text{ (TD error)}}] \nabla_w Q_w(s_t,a_t)$



Problem: Instability